CS440 Project 1

Zhitong Guo
Jinglei Chen
Rutgers University
CS440: Introduction to Artificial Intelligence

Fall 2026

1 Introduction

This project studies path planning in a dynamic environment where a bot must navigate through a randomly generated ship while a fire spreads over time. The bot begins at a randomly selected open cell, a fire begins at another open cell, and a button that activates the fire suppression system is placed at a third open cell. The bot succeeds if it reaches the button before being reached by the fire.

The main objective of the project is to design and compare several navigation strategies under different values of the ship flammability parameter q. Four bot strategies were implemented. The first three follow the specifications given in the assignment, while Bot 4 uses a custom risk-aware path planning strategy.

The experiments focus on how the success probability of each bot changes as q increases, when different strategies begin to make different decisions, and why particular strategies succeed or fail in different environments.

2 Environment and Simulation

2.1 Ship Generation

The ship is represented by a $D \times D$ grid. Each cell is either open or blocked. Movement is allowed only between open cells in the four cardinal directions: up, down, left, and right.

The ship is generated by first selecting a random interior cell to open. Additional blocked cells are then opened when they have exactly one open neighbor. This process continues until no more such cells are available.

After the initial layout is produced, dead ends are identified. A dead end is an open cell with exactly one open neighbor. Random blocked neighbors of dead ends are opened until the total number of dead ends has been reduced by at least half.

For the experiments in this report, the ship size was

$$D = 40. \tag{1}$$

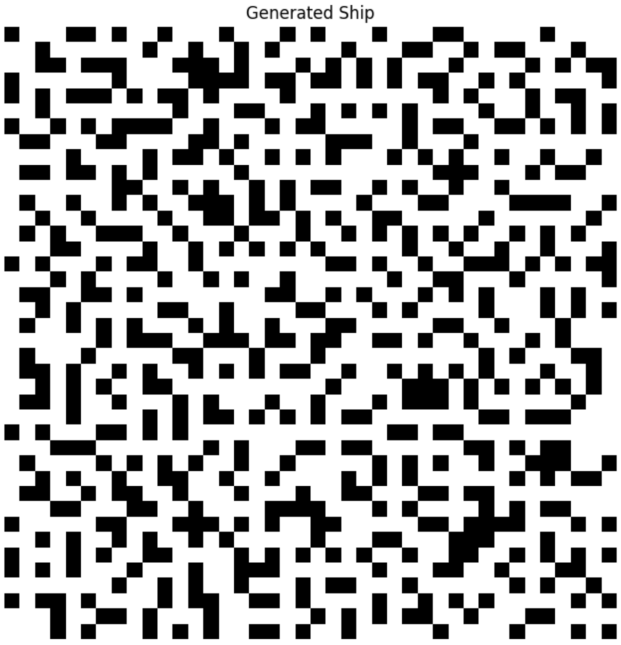


Figure 1: Example of a randomly generated ship environment with $D = 40$.

2.2 Bot, Button, and Initial Fire

At time $t = 0$, the bot, button, and initial fire are randomly placed on three distinct open cells.

The symbols used in the visualizations are:

Object	Symbol
Bot	*
Fire	x
Button	o

The three starting positions are sampled simultaneously from the set of open cells to guarantee that they are distinct.



Figure 2: Initial task configuration showing the bot, button, and initial fire on three distinct open cells.

2.3 Fire Spread

The fire begins at one randomly selected open cell. At every time step, each currently non-burning open cell can catch fire.

The probability that a cell catches fire is

$$P(\text{catch fire}) = 1 - (1 - q)^K, \tag{2}$$

where q is the flammability parameter and K is the number of currently burning neighboring cells.

When q is small, the fire spreads slowly. As q increases, the probability of ignition increases. A cell with several burning neighbors is also more likely to catch fire than a cell with only one burning neighbor.

All new fire cells for a time step are calculated before the fire state is updated. This prevents a newly burning cell from spreading the fire again during the same time step.

2.4 Task Execution

Each time step follows the same sequence:

1. The bot decides which action to take.
2. The bot moves to an adjacent open cell or stays in place.
3. If the bot reaches the button, the task succeeds immediately.
4. Otherwise, the fire spreads once.

5. If the bot and fire occupy the same cell, the task fails.

The main difference between the four bots is therefore how the next action is selected.

3 Bot Strategies

3.1 Common Shortest-Path Search

Bots 1, 2, and 3 are based primarily on shortest-path search.

Breadth-first search (BFS) is used because every legal movement has equal cost. BFS explores the grid level by level and therefore returns a shortest path when one exists.

A shared BFS implementation is used by multiple strategies so that the differences between bots come from their decision rules rather than from different implementations of shortest-path search.

3.2 Bot 1

Bot 1 plans a shortest path from its starting location to the button at time $t = 0$. The initial fire cell is treated as blocked during this planning step.

After the initial path is computed, Bot 1 follows the path without replanning.

This makes Bot 1 computationally simple, but it does not react to changes in the environment. If the fire later spreads onto or near its original route, Bot 1 continues following the same plan.

3.3 Bot 2

Bot 2 replans at every time step.

Before each move, it calculates a new shortest path from its current location to the button while treating all currently burning cells as blocked.

Only the next step of the newly calculated path is executed. At the next time step, the process is repeated using the updated fire configuration.

Bot 2 therefore reacts to the current fire, unlike Bot 1. However, it only considers cells that are already burning and does not explicitly account for cells that may become dangerous in the near future.

3.4 Bot 3

Bot 3 is more conservative.

At every time step, it first attempts to find a shortest path while avoiding both:

- all currently burning cells, and
- all open cells directly adjacent to a currently burning cell.

If no such path exists, Bot 3 falls back to the Bot 2 rule and searches for a shortest path while avoiding only the currently burning cells.

This strategy attempts to provide a safety buffer around the fire while still allowing the bot to make progress when a completely buffered route does not exist.

3.5 Bot 4: Risk-Aware Path Planning

3.5.1 Design Motivation

Bot 4 was designed to balance two competing objectives:

1. reaching the button quickly, and
2. remaining far enough from the fire to reduce risk.

Unlike Bot 3, Bot 4 does not classify cells near the fire as simply safe or unsafe. Instead, it assigns different movement costs to cells based on their estimated fire risk.

This allows Bot 4 to accept a slightly longer route when the additional distance provides greater safety, while still allowing it to pass through moderately risky areas when avoiding them would require an excessively long detour.

3.5.2 Fire Distance Estimation

A multi-source BFS is used to compute the distance from every reachable open cell to the nearest currently burning cell.

The search begins simultaneously from all current fire cells. This produces the shortest distance through the actual ship layout.

Using BFS distance is important because Manhattan distance alone ignores walls. Two cells may appear geographically close while being separated by blocked cells and therefore far apart in terms of actual fire propagation.

Let

$$d_{\text{fire}}(c) \tag{3}$$

represent the BFS distance from cell c to the nearest current fire cell.

3.5.3 Risk-Aware Cell Cost

Each candidate cell is assigned a movement cost containing both a normal movement term and a fire-risk term.

The cost used in the implementation is

$$\text{cost}(c) = 1 + \frac{\lambda(q)}{d_{\text{fire}}(c) + 1}. \tag{4}$$

The first term represents the normal cost of taking one step.

The second term penalizes cells that are close to the fire. In the current implementation,

$$\lambda(q) = 10q. \tag{5}$$

Therefore, a larger value of q increases the penalty for approaching the fire.

This reflects the intuition that when the fire spreads rapidly, the bot should place greater emphasis on safety. When the fire spreads slowly, taking a shorter route becomes relatively more attractive.

The constant factor 10 was not derived as a theoretical optimum. It was chosen as a simple scaling factor large enough for fire distance to influence route selection without making the fire-risk term completely dominate path length. This choice is therefore a design parameter and represents one limitation of the current Bot 4 implementation.

3.5.4 Path Planning

Dijkstra's algorithm is used to find a minimum-cost route under the risk-aware cost function.

At every time step, Bot 4:

1. observes the current burning cells,
2. computes the fire-distance map,
3. assigns risk-aware costs to open cells,
4. computes the minimum-cost route to the button,
5. executes only the next move,
6. repeats the calculation at the next time step.

Thus, Bot 4 continuously adapts to the current fire while making a tradeoff between route length and safety.

3.5.5 Computational Efficiency

The fire-distance map is computed using one multi-source BFS rather than running a separate search from every cell. For a grid represented as a graph, this requires $O(V + E)$ time, where V is the number of open cells and E is the number of connections between neighboring open cells.

Dijkstra's algorithm is then used to compute the risk-aware route. With the priority queue used in the implementation, this requires approximately $O((V + E) \log V)$ time. The implementation also reuses shared neighbor and path-search logic across strategies where possible, reducing duplicated computation and code.

4 Experimental Setup

The four bots were evaluated using randomly generated 40×40 ships.

For each test configuration, all four bots were evaluated using the same ship and the same initialization seed so that the starting bot position, button position, initial fire position, and random fire progression were comparable.

The primary evaluation metric was success rate:

$$\text{Success Rate} = \frac{\text{Number of Successful Trials}}{\text{Total Number of Trials}}. \tag{6}$$

Two stages of experimentation were performed.

4.1 Broad Exploration

The first experiment evaluated

$$q = 0.1, 0.2, \ldots, 0.9. \tag{7}$$

For each value of q, 50 trials were performed.

The purpose of this experiment was to identify the general relationship between flammability and bot performance and to determine the range of q values where the strategies showed the clearest differences.

4.2 Detailed Experiment

Based on the broad experiment, the range

$$0.30 \leq q \leq 0.70 \tag{8}$$

was selected for more detailed testing.

The tested values were

$$q = 0.30,\ 0.35,\ 0.40,\ 0.45,\ 0.50,\ 0.55,\ 0.60,\ 0.65,\ 0.70. \tag{9}$$

For each value of q, 200 trials were performed.

This produced a larger data set for comparing relatively small differences between the four strategies.

5 Results

5.1 Broad Flammability Experiment

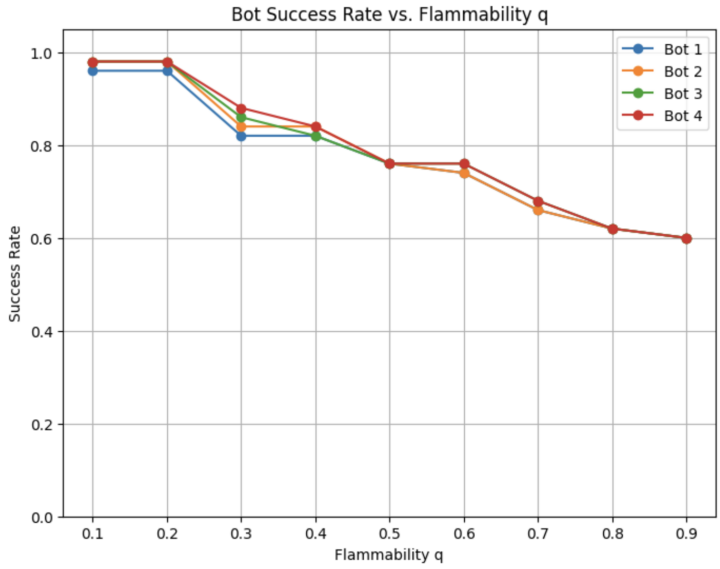


Figure 3: Success rate of Bots 1–4 across the broad range of flammability values on randomly generated 40×40 ships. Each point represents 50 trials.

The broad experiment shows a clear overall decrease in success rate as q increases.

At low values of q, the fire spreads slowly enough that all four bots succeed in most environments. The differences between strategies are therefore small.

At very high values of q, the fire spreads rapidly and the strategies again become increasingly similar in performance. In many such environments, the bot has limited opportunity to react regardless of the planning strategy.

The largest visible differences occurred in the intermediate region, motivating a more detailed experiment between approximately $q = 0.30$ and $q = 0.70$.

5.2 Detailed Analysis of the Interesting q Range

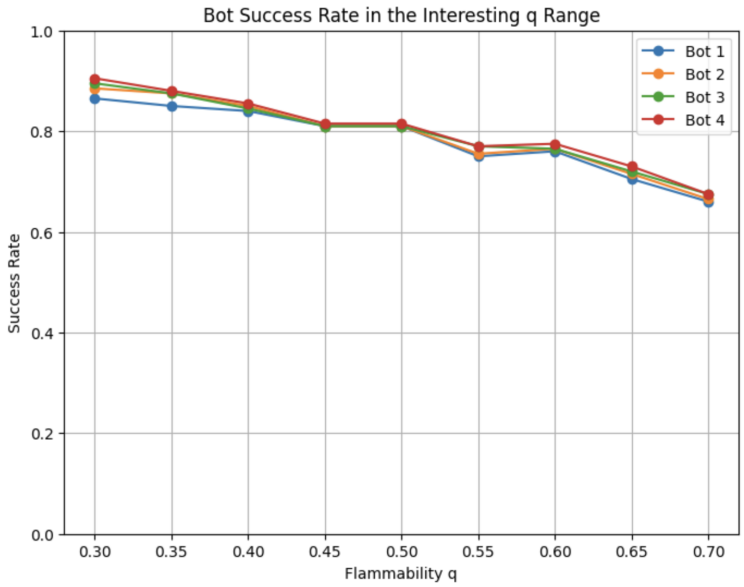


Figure 4: Detailed comparison of Bot 1–4 success rates in the most informative range of q on randomly generated 40×40 ships. Each point represents 200 trials.

The detailed experiment confirms the general downward relationship between q and success probability.

Bot 1 generally has the lowest or near-lowest success rate. This is consistent with its inability to react to changes in the fire after its initial route has been selected.

Bots 2 and 3 generally improve on Bot 1 by using the current fire configuration during planning.

Bot 4 achieves the highest or near-highest success rate for several tested values of q. For example, at $q = 0.30$, the observed success rates were:

Bot	Success Rate
Bot 1	0.865
Bot 2	0.885
Bot 3	0.895
Bot 4	0.905

This provides an example where the risk-aware strategy outperformed the three baseline strategies.

However, the improvement is relatively small. The results do not support the claim that Bot 4 is universally superior. Instead, Bot 4 provides a modest advantage in some intermediate-risk environments.

Because each value of q is estimated from a limited number of random trials, small changes are expected. For example, the slight increase in success rate from $q = 0.55$ to $q = 0.60$ is likely due to random variation and does not mean that a more flammable ship is safer.

Table 1: Success rates from the detailed experiment on 40×40 ships, based on 200 trials for each value of q.

q	Bot 1	Bot 2	Bot 3	Bot 4
0.30	0.865	0.885	0.895	0.905
0.35	0.850	0.875	0.875	0.880
0.40	0.840	0.850	0.845	0.855
0.45	0.810	0.810	0.810	0.815
0.50	0.810	0.810	0.810	0.815
0.55	0.750	0.755	0.770	0.770
0.60	0.760	0.765	0.765	0.775
0.65	0.705	0.715	0.720	0.730
0.70	0.660	0.665	0.675	0.675

6 Analysis

6.1 Performance Comparison

The experiments show three broad performance regions.

First, when q is small, the fire spreads slowly. Most bots have enough time to reach the button even when their paths are not particularly cautious. As a result, the difference between the strategies is limited.

Second, at intermediate values of q, path planning becomes more important. The fire spreads quickly enough to punish poor route choices but not so quickly that success becomes largely determined by the initial configuration. This is the region where the differences between Bots 1–4 are most informative.

Third, when q becomes large, the fire spreads so aggressively that the bots increasingly converge in performance. In these configurations, even sophisticated replanning may not create enough additional time to escape.

The data therefore support the expectation that the value of an improved strategy is greatest in an intermediate range of fire intensity.

6.2 Failure Analysis

The failure behavior of each bot reveals limitations in its decision rule.

6.2.1 Bot 1

Bot 1 can fail because its path is fixed at the beginning of the simulation.

A route that is safe at $t = 0$ may later become occupied by fire. Since Bot 1 does not replan, it can continue toward a region that has become dangerous even when an alternative route exists.

This behavior explains why Bot 1 generally performs worse than the adaptive strategies.

6.2.2 Bot 2

Bot 2 responds to current burning cells, but it does not explicitly consider future risk.

A cell can be legal and non-burning at the current time while being very close to several burning cells. Bot 2 may select such a cell because it is part of the shortest current path.

The fire may then spread into that location before the bot can escape.

6.2.3 Bot 3

Bot 3 attempts to avoid this problem by avoiding cells adjacent to the fire.

However, this rule can sometimes be overly conservative. Avoiding all adjacent cells may force the bot to take a much longer route.

A longer route gives the fire additional time to spread. Therefore, being more cautious at the local level does not always produce a safer global route.

6.2.4 Bot 4

Bot 4 attempts to address this tradeoff by using continuous risk costs rather than a binary safe/unsafe classification.

However, the strategy is still imperfect.

The current risk estimate is based mainly on the present distance from the fire. It does not explicitly predict the exact future fire state at the time the bot eventually reaches a cell.

The weighting function can also influence performance. If the risk penalty is too large, the bot may take unnecessarily long detours. If it is too small, its behavior can become similar to Bot 2.

Some failures are decision-dependent while others are effectively unavoidable. For Bot 1, an alternative route can sometimes exist after the original route becomes dangerous, but Bot 1 cannot exploit it because it never replans. For Bots 2–4, a different move can sometimes provide a safer route, especially when the current decision underestimates future fire risk or takes an excessive detour. In other configurations, however, the fire can cut off every useful route to the button; in those cases, no local decision would be sufficient to save the bot.

6.3 Cases Where Bots Make Different Decisions

An important diagnostic question is whether the more complex strategies actually produce different behavior.

If Bot 4 always selected the same path as Bot 2, the additional risk calculations would provide little practical benefit.

The diagnostic search in the notebook found a concrete example at seed 8 with $q = 0.5$. From the same state, Bot 2 selected cell $(19, 0)$, while Bot 4 selected cell $(18, 1)$.



Figure 5: A diagnostic example in which Bot 2 and Bot 4 choose different next moves from the same state.

Figure 5 shows why the two strategies can diverge. Bot 2 chooses according to the shortest path that avoids the currently burning cells. Bot 4 evaluates the same state using its risk-aware path cost, so a move that lies on the shortest route can receive a larger cost when it is closer to the fire. The different next moves therefore confirm that Bot 4 is not simply reproducing Bot 2. This diagnostic check establishes a difference in decision making; it does not by itself establish that either move ultimately produces a successful run.

A useful future extension would be to record the selected path for each strategy at every time step and directly visualize the first point at which the bots make different decisions.

6.4 Towards an Ideal Bot

An ideal bot would reason not only about where the fire is currently located, but also about where the fire is likely to be when the bot reaches future cells.

One approach would be to estimate two quantities for every candidate cell:

1. the number of steps required for the bot to reach the cell, and
2. the expected number of steps required for the fire to reach the same cell.

A cell would become dangerous when the estimated fire arrival time approaches or becomes smaller than the bot arrival time.

A more advanced strategy could simulate multiple possible future fire progressions and estimate the probability that each route remains safe.

However, more sophisticated prediction requires more computation.

In a realistic dynamic environment, computation itself has a cost because the environment may

continue changing while a decision is being made. Therefore, an ideal agent should balance planning accuracy with computational efficiency.

When the button is nearby, the fire is spreading rapidly, or the number of safe alternatives is very small, it may be more effective to commit quickly to a route rather than spend additional time searching for a theoretically optimal plan.

7 Discussion

7.1 Design Choices

The main design choice for Bot 4 was to replace the hard safety boundary used by Bot 3 with a graded risk model.

Bot 3 treats a cell adjacent to fire as completely unavailable whenever an alternative buffered path exists. In contrast, Bot 4 allows such a cell to remain available but assigns it a larger cost.

This was intended to create a more flexible balance between path length and safety.

The use of actual BFS fire distance rather than Manhattan distance was also important because the ship contains walls. Fire risk should therefore depend on reachable distance through the ship rather than only geometric distance.

Finally, the risk weight was linked to q because the appropriate level of caution depends on the speed of fire spread.

7.2 Unexpected Results

One initially expected outcome was that the adaptive strategies would outperform Bot 1.

This generally occurred, but the differences between the four strategies were smaller than expected in many environments.

Bot 4 did not universally outperform the other bots. In several regions, Bots 2, 3, and 4 achieved nearly identical success rates.

This suggests that the structure of the environment and the initial positions often have a stronger effect on the final outcome than relatively small differences in planning strategy.

It also shows that a more complicated algorithm does not automatically produce a large performance improvement.

7.3 Limitations

Several limitations remain in the current approach.

First, Bot 4 uses current fire distance as a proxy for future risk rather than directly predicting future ignition probabilities.

Second, the function

$$\lambda(q) = 10q \tag{10}$$

was chosen as a simple relationship between flammability and risk sensitivity. Other functions may provide better performance.

Third, success rate alone does not describe all aspects of bot behavior. Two bots can have similar success rates while producing very different path lengths, computational costs, or failure patterns.

Future work could therefore evaluate additional metrics such as average number of time steps, path length, computation time, and distance from fire.

8 Bonus: Ship Layout Optimization

For the bonus task, a local-search procedure was implemented to search for a safer ship layout. Each candidate was produced by swapping one open cell with one blocked cell. A candidate was accepted only when all open cells remained connected, and the total number of open cells was kept unchanged.

The search evaluated each layout using the average success rate of Bots 1–4 at $q = 0.3$, $q = 0.5$, and $q = 0.7$. It ran for 10 generations with 20 candidates per generation and 20 trials for each bot and value of q. The best layout reached a fitness of 0.9167 on the trials used during the search.

To check whether this improvement generalized to new simulations, the selected layout and the original layout were then evaluated using a separate seed and 200 trials for every bot and value of q. Table 2 reports the success rates averaged across the four bots.

Table 2: Independent comparison of the original and optimized ship layouts. Each value is averaged across Bots 1–4 using 200 trials per bot.

q	Original Ship	Optimized Ship
0.30	0.8525	0.8275
0.50	0.7088	0.7175
0.70	0.6188	0.6125
Overall	0.7267	0.7192

The optimized layout showed a small improvement at $q = 0.5$, but it performed slightly worse at $q = 0.3$ and $q = 0.7$. Its overall success rate was 0.7192, compared with 0.7267 for the original layout. Therefore, the bonus search did not produce a clear or substantial improvement on the independent trials.

The large difference between the search fitness and the independent result suggests that the search adapted too closely to the limited random trials used during optimization. More training trials, a larger variety of seeds, or a fitness function based partly on structural properties of the ship could make the layout search more reliable. The result is still useful because it shows that improving performance on the trials used during a search does not necessarily produce a generally safer ship.

9 Conclusion

This project compared four navigation strategies for reaching a fire-suppression button in a dynamically spreading fire environment. This experiments show that success rate generally decreases as the ship flammability parameter q increases.

Bot 1 is limited by its inability to adapt after its initial plan is created. Bots 2 and 3 improve on this behavior through repeated replanning. Bot 4 introduces a risk-aware cost model that balances path length with distance from the current fire. The strategy achieved the highest or near-highest success rate for several intermediate values of q, although the improvement over Bots 2 and 3 was generally modest.

The results demonstrate that the most interesting differences between strategies occur at intermediate fire intensities. When the fire spreads very slowly, almost all reasonable strategies succeed. When it spreads extremely quickly, even sophisticated planning has limited ability to improve the outcome.

Overall, the project illustrates an important principle in dynamic planning: the shortest path is not always the best path, but increased caution also has a cost. Effective navigation requires balancing immediate progress, future risk, and computational complexity.

The bonus layout search also showed that the tested optimization method did not create a clearly safer ship under independent evaluation. Its small improvement at one value of q was offset by slightly lower performance at the other tested values.

10 Contributions

Zhitong Guo: Implemented the remaining project components, including the button, task simulation, bot strategies, experiments, and analysis, and contributed to the remaining report writing and revision and bonus part.

Jinglei Chen: Implemented the first three project components—ship generation, bot functionality, and fire simulation—and contributed to report writing.

Both group members contributed to both implementation and preparation of the final writeup.

Project collaboration: The project was managed through the GitHub repository. GitHub was used for version control and repository management, while Overleaf was used for collaborative report writing.